The 2D LADMM Solver on a GPU: Compiled Iteration, Parallel Projection, Mixed Precision

Optimal Transport / Schrödinger Bridge Project
Companion to the Python/JAX 2D code in `python/2d/setup`

September 2026

Abstract

This note records what was changed in the 2D Python/JAX solver to make it fast on an NVIDIA GH200, why each change matters, what was measured, and which precision choices are safe. It is written so the same decisions can be carried to the 3D code. Sections 1–5 cover speed and memory in double precision; Section 6 covers single and mixed precision, including the one non-obvious result: the algorithm tolerates float32 storage of a variable only if the optimality condition attached to that variable is evaluated at the stored value. Section 7 describes the DeltaAI node, which parts of the code run where, what is serial, and when several GPUs would pay off.

1 The algorithm and the cost model

One LADMM iteration for $\min_{x,y} I_{\mathrm{FP}}(x) + \mathrm{KE}(y)$ s.t. $Ax = y$ is

$$r^k = Ax^k - y^k, \tag{1}$$

$$x^{k+1} = P\Big(x^k - \tfrac{1}{\tau}A^\top(\gamma r^k - \delta^k)\Big), \tag{2}$$

$$y^{k+1} = \mathrm{prox}_{\mathrm{KE}/\gamma}\Big(Ax^{k+1} - \delta^k/\gamma\Big), \tag{3}$$

$$\delta^{k+1} = \delta^k - \gamma\,(Ax^{k+1} - y^{k+1}), \tag{4}$$

where P is the projection onto the discrete Fokker–Planck constraint (one DCT in (x, y), a tridiagonal solve in t per spatial mode, one inverse DCT), A is the staggered-to-cell-centre interpolation, and the prox of $\mathrm{KE}(\rho, m) = |m|^2/(2\rho)$ is a pointwise cubic. Every operation is either a transform, a banded solve, or an elementwise map over arrays of $n_t n_x n_y$ entries; nothing is a dense matrix.

On a GPU the cost of such a loop has three parts, and it is worth knowing which one you are paying:

1. **Launch latency.** Every kernel costs 5–20 μs to dispatch, and a device-to-host copy (a `float()` on an array) stalls the pipeline. An iteration written as ∼300 small `jnp` operations pays ∼300 launches whether the arrays hold 10^3 or 10^7 entries.

2. **Memory traffic.** Elementwise maps, interpolations and reductions are bound by bytes moved, not flops. A GH200 moves about 4 TB/s; a float64 array of 256^3 entries is 134 MB, so each pass over the state costs ∼35 μs.

3. **Arithmetic.** Only the DCTs (as dense matrix products) and the cubic in the prox have appreciable flop counts. On H100/GH200 the float64 tensor-core rate (67 TFLOP/s) equals the non-tensor float32 rate, so a float64 matmul is not slower than a float32 one at the precision this code needs (Section 3.2).

The eager code was entirely in regime 1: 215 ms per iteration at 32^3 and 185 ms at 128^3, i.e. flat in the grid size (Table 1). Everything in Sections 2–3.2 is about moving from regime 1 to regime 2.

2 Compiling the iteration

`ladmm_jit.py` traces one whole iteration (1)–(4), including the seven residual and scale norms used by the stopping test, into a single XLA executable, and runs `check_every` iterations back to back inside a `lax.scan`. The only host synchronisation is the copy of the $k \times 9$ array of scalar diagnostics at the end of each chunk. Four details make this work.

Everything traceable. No `float()` inside the loop: the norm callbacks return `jnp` scalars, and the stopping test runs on the host on the chunk's diagnostics. `State2D` and the projection's precompute dataclasses are registered as pytrees (`jax.tree_util.register_dataclass`) so they pass through `jit` and `scan` as arguments rather than being baked in as constants.

One fewer A per iteration. The residual (1) equals the D of the previous iteration, so it is carried instead of recomputed. Bitwise the same numbers.

Diagnostics are not free. Each norm is a full reduction over the state; on a GPU the seven of them cost about as much as the KE prox. `diag_level="stopping"` drops the three descriptive ones ($\|x^{k+1} - x^k\|$ etc.).

Recompilation. The compiled chunk closes over the pipeline's callables, so every call to `discretize_then_optimize_*` recompiles (0.4–0.6 s on the GH200). A benchmark that times a fresh 50-iteration solve reports ∼10 ms/iteration of pure compile time. Measure with `walltime - walltime_first_chunk`, as `bench_2d.py` does.

With `check_every=1` the iterates equal the eager solver's to rounding (XLA may fuse $ab + c$ into an FMA). With `check_every=k` the stop is detected up to $k - 1$ iterations late; every extra iteration is a further LADMM step, so this only moves the answer closer.

3 The projection on a GPU

3.1 The time solve: sequential Thomas vs. parallel DCT

After the DCT in (x, y) the projection is $n_x n_y$ independent tridiagonal systems in t, one per mode (k_x, k_y), all with the same structure: constant interior diagonal, constant off-diagonal e, and two deviating end rows. The MATLAB and the eager Python code solve them with a batched Thomas sweep, vectorised over modes. Written as `lax.scan` (`thomas_batch_precomp/solve`) rather than a Python list, the sweep is $2n_t$ tiny sequential kernels per projection; on a GPU this

is latency-bound again and is the reason the uniform-grid projection was 2–3× slower than it needed to be.

The parallel alternative (`method="dct"`, `precomp_dct_modes`) uses the fact that each per-mode matrix is

$$T = T_0 + a_0\, e_0 e_0^\top + a_1\, e_{n_t-1} e_{n_t-1}^\top,$$

with T_0 a Neumann Laplacian in t, diagonalised by the DCT-II in t with eigenvalues $\mu_j = \text{mid} + 2e\cos(\pi j/n_t)$, and (a_0, a_1) the two end-row deviations. By the Woodbury identity [2],

$$T^{-1} = T_0^{-1} - T_0^{-1}U\,(I + C\,V^\top T_0^{-1}U)^{-1} C\,V^\top T_0^{-1},$$

with $U = V = [e_0\ e_{n_t-1}]$ and $C = \text{diag}(a_0, a_1)$: a 2×2 capacitance matrix per mode, precomputed once. A solve is then one DCT in t, a division by μ_j, one inverse DCT, and a rank-2 elementwise correction: no sequential dependency at all. The DC mode $(0, 0)$ needs no special case: there $a_0 = a_1 = 0$ and $\mu_0 = 0$, and zeroing $1/\mu_0$ is exactly the pseudo-inverse the Thomas path applied by hand. The non-uniform time grid is not Toeplitz in t and keeps the scan-Thomas path; it is correspondingly slower (Table 1).

3.2 The DCT kernel

JAX has no DCT. Three implementations of the orthonormal DCT-II along one axis are provided (`set_dct_method`): mirror-and-FFT of length $2N$, Makhoul's N-point reordering [3], and a dense $N \times N$ cosine matrix applied with one `tensordot`. On the GH200 the matrix form is the fastest for $N \le 512$ by 1.3–1.9×: XLA's FFT along a non-last axis inserts transposes, whereas the matmul is a single cuBLAS call on an axis of the caller's choosing. Its cost is $O(N)$ per entry instead of $O(\log N)$, which does not matter at these sizes. In float32 the matmul precision must be pinned (`jax_default_matmul_precision="highest"`) or it silently uses TF32 tensor cores with a 10-bit mantissa.

4 Measured speed

grid $n_t = n_x = n_y$	eager	jit, scan-Thomas	jit, DCT modes	clustered (scan)
32^3	215	0.25	0.09	0.6
128^3	185	2.5	0.87 (CN) / 1.04 (ETD)	3.4
256^3	–	14.8	6.0–7.6	12.3

Table 1: Milliseconds per LADMM iteration, float64, GH200, steady state (compile excluded), `check_every`=50–100. "DCT modes" uses the matmul DCT. The clustered time grid cannot use the DCT-in-t solver.

Ordered by effect: compiling the loop (100–2000×) ≫ DCT modes vs. scan-Thomas (2–3× at 32^3–128^3) > matmul DCT vs. FFT (1.3–1.9× at $\ge 128^3$) > `check_every` 10–50 vs. 1 (2× at 32^3, 10% at 128^3) > `diag_level="stopping"` (5–10%). The 28-run grid-convergence suite (Gaussian, 7 values of ε, $n = 32..256$, 40k–150k iterations) takes 159 min of solve time in float64.

5 Memory footprint

The solver holds about 67 arrays of size $n_t n_x n_y$ in float64 (9.0 GB at 256^3, 134 MB per array). They are: the LADMM carry (x, y, By, δ, D) with three components each (15), the previous x inside the scan (3), the projection's temporaries (the $(n_t + 1)$-plane DCT of ρ, the residual, the mode solution, its DCT in t, the weighted potential and its derivatives: $\sim$10), the precompute ($1/\mu$, g_0, g_1, D_{mod}: 4), the reference solutions for error tracking (3 per reference), the diagnostics' temporaries, and XLA's own scratch. Only the first three groups are essential; the references can be kept off the device and, with $\alpha = 1$, the previous y and By are never read. This is the accounting behind the 3D estimates in Section 8.

6 Precision

Float32 halves every byte in Section 5. It is also where the algorithm turns out to be fragile, in a specific and instructive way.

6.1 Plain float32

Same 28 runs, everything float32 (setup, precompute and references built in float64 and cast): 1.05× faster at 32^3, 1.5–1.6× at $\geq 128^3$ uniform, 1.2–1.3× clustered (the sequential sweep does not care about bytes), 2.2× less memory. Accuracy: identical to float64 for $n \leq 64$; at $n = 256$ and small ε the density error floors at 10^{-3} instead of 3×10^{-5}; mass conservation degrades from 10^{-13} to 10^{-4}–10^{-2}; on the clustered grid at $n = 256$ the solve diverges for $\varepsilon \geq 0.1$.

6.2 What needs float64: three hypotheses, one survivor

The projection. R and $R^\top$ are time differences divided by Δt. The residual's rounding is harmless (the solve smooths it: the $1/j^2$ tail of the time spectrum sums to a constant), but $R^\top$ applied to the large, smooth potential turns float32 rounding into an $O(u_{32}\,|\phi|/\Delta t)$ error, and on the clustered grid the N-weighted per-mode systems span $1/(\Delta t_{\min}^2 \Delta x^2)$ to $1/\Delta t_{\max}^2$. Running the projection in float64 on a float32 iterate (`precision.mixed_projection`: cast up, project, cast down) restores mass conservation to 10^{-9} and removes the divergence – but the error floor stays: 6.1×10^{-4} at $n = 128$ against float64's 3.4×10^{-4}, and 4.3×10^{-4} at $n = 256$ against 3.2×10^{-5}, with the residuals D, P identical to float64's.

Sub-ulp steps. Late LADMM steps are $P/\tau \sim 10^{-8}$ relative, below float32's unit roundoff $u_{32} = 6 \times 10^{-8}$, so a float32 x cannot represent them. Keeping x and δ as float64 "master copies" (`LadmmConfig.master_dtype`), the recipe of mixed-precision training [8], changed nothing: 5.6×10^{-4} at $n = 128$.

Frozen inconsistency in the KKT system. Two diagnostic runs, each float64 everywhere except for one injected rounding (`--precision fp64rx`, `fp64px`): rounding x to float32 before A gives 3.377×10^{-4}, float64's value to four digits; running only the KE prox and y in float32 gives 5.98×10^{-4}, the whole floor. The reason is the identity, exact for the prox of a differentiable function,

$$\delta^{k+1} = \delta^k - \gamma\,(Ax^{k+1} - y^{k+1}) = -\nabla \mathrm{KE}\Big(y^{k+1}\Big)\,, \tag{5}$$

which holds at the *exact* prox output. When y is stored in float32 the dual is the KE gradient at a point u_{32} away from the y the constraint $Ax = y$ actually uses. That is a frozen, non-random inconsistency in the KKT system, and it is amplified by the ill-conditioned directions of the problem – the same slow modes that make LADMM need 10^5 iterations – by roughly 10^3. The excess error sits in the core of the density, mid-trajectory, with the shape of the slowest mode, not in the tails. Rounding x has no such inconsistency: the prox is evaluated at the same rounded x the constraint sees, so the KKT conditions hold at the stored iterate and the rounding only selects a nearby solution.

Rule 1. *A variable may be stored in reduced precision only if every optimality condition involving it is evaluated at the stored value. Storage precision is a property of the fixed point, not of the arithmetic.*

6.3 The working scheme (`--precision mixedc`)

- float32: y, By, D, the consensus map A applied to the rounded x, the reference-error diagnostics.
- float64: x, δ, the projection, $A^\top$, and the *arithmetic* of the KE prox (its output is stored float32). The float32 Cardano root is up to 7×10^{-4} off in near-empty cells; in float64 the casts fuse into the elementwise kernel and cost nothing.
- The dual is computed from the stored y by (5): $\delta^{k+1} = -\nabla \mathrm{KE}(y_{32}^{k+1})$ in float64, with the standard update kept in the cells the prox clamps to $\rho = 0$ (`ladmm_jit.py`, `dual_fn`).

Results over the 28 runs: every error agrees with float64 to four digits, the distance between the two solutions is at most 6×10^{-7}, fitted orders 3.68 / 3.68 / 2.52 / 2.02 / 2.05 / 1.87 / 1.02 for $\varepsilon = 0, 10^{-8}, \ldots, 1$ (float64's), mass conservation float64's. Speed is 0.8–1.1× float64 (1.04× at 256^3); peak memory is 60% of float64 at 128^3 and 71% at 256^3 (6.4 GB vs. 9.0 GB). The scheme is a memory tool, not a speed tool: only the cheap, memory-bound parts became float32, and the float64 matmuls were never the bottleneck.

6.4 Half precision

Two directions are common in the literature: compute in a low precision and accumulate in a higher one (mixed-precision training [8]: float16 arithmetic, float32 master weights), and factorise in a low precision and refine the residual in a higher one (iterative refinement in three precisions [9, 10]). Neither helps here.

- The compute-low direction is what Sections above tested with float32 as the low precision: the fixed point moved by $u \times 10^3$. With float16 ($u = 5 \times 10^{-4}$) or bfloat16 ($u = 4 \times 10^{-3}$) even the consistent scheme would place the solution $\sim u$ away from the true one, i.e. 10^{-3}–10^{-2} relative, above the discretization error at every $n \geq 64$. The residual $D = Ax - y$ is a cancellation of $O(64)$ numbers; in half precision it is noise at the 10^{-2} level and the dual update (4) integrates that noise. Float16's range (6×10^{-5} to 6.5×10^4 normal) also flushes the density's tails to zero.
- The refine direction needs a cheap low-precision approximate solve and an exact residual. The projection is already a direct $O(n)$ solve; a half-precision version plus a float64 residual costs more than the float64 solve it would replace.

- Half-precision DCTs on tensor cores would put a 10^{-3} relative error into the projection's residual; this is why even TF32 is disabled.
- The memory argument does not hold either: the float32 arrays in `mixedc` are ∼9 of ∼48 array-equivalents. Halving them saves 5%. The bytes are in the float64 state, the projection's temporaries and the diagnostics.

7 The DeltaAI node: where the work runs

7.1 Hardware

A `ghx4` node is four NVIDIA GH200 superchips. Each superchip is a 72-core Grace CPU (Arm Neoverse-V2, ∼120 GB LPDDR5X) and an H100 GPU with 98 GB of HBM3 (`nvidia-smi` reports 97,871 MiB usable), joined by NVLink-C2C at 900 GB/s. The four GPUs of a node are connected by NVLink; nodes are connected by HPE Slingshot, more than an order of magnitude slower than NVLink. A job asks for a share of one node: the suite runs used `--gpus=1 --cpus-per-task=16 --mem=96g`, i.e. one GPU and 16 of the 288 cores. The login nodes are the same Grace CPUs without a GPU.

7.2 What runs on the CPU and what on the GPU

- **CPU (Python, NumPy, SciPy):** the driver script, the problem setup, the 1D Sinkhorn reference fits and their spectral resampling, the stopping test on each chunk's diagnostics, the per-time-row error rows and the `.npz` output after the solve. All of this is a few seconds per run; the solve itself is minutes.
- **GPU (XLA):** the precompute (built with `jnp`, so it already lives on the device), the initial iterate, and every operation inside the compiled chunk: A, $A^\top$, both projections, the KE prox, the dual update, the norms, the reference errors. The Python interpreter's only job during a solve is to call the chunk `max_iter/check_every` times and read back a $k \times 9$ array.

The same compiled code runs on the 72-core Grace by setting `--device cpu` (Table 2); the GPU is 20–60× faster, and the gap grows with the grid because the CPU is bound by DRAM bandwidth (∼500 GB/s) against HBM3's 4 TB/s.

grid	Grace CPU, 72 threads	one GH200 GPU	ratio
64^3 uniform, DCT modes	5.7	0.27	21
128^3 uniform, DCT modes	36.7	1.15	32
256^3 uniform, DCT modes	423	7.3	58
64^3 clustered, Thomas	5.9	1.25	5
128^3 clustered, Thomas	39.6	3.07	13

Table 2: Milliseconds per iteration, ETD (CN for the clustered rows), float64, same compiled code on both devices (`cpu_vs_gpu.sbatch`). The sequential Thomas sweep hurts the GPU more than the CPU: the ratio drops from 32 to 13 at 128^3.

7.3 Serial and parallel parts

Within one iteration everything is parallel over the grid except two things, and the iteration count itself is serial.

- **Parallel over all $n_t n_x n_y$ entries:** A, $A^\top$, the residual and the corrections inside the projection, the KE prox, the dual update. Each is one or a few fused elementwise kernels.
- **Parallel over $n_x n_y$ modes, sequential over t:** the Thomas sweep ($2n_t$ dependent steps; the clustered grid). The DCT-in-t solver removes this: it is parallel over all entries.
- **Transforms:** the DCTs are dense matrix products, parallel over everything, executed by cuBLAS.
- **Reductions:** the norms are tree reductions over the state; parallel, but each one is a full pass over memory and ends in a single scalar.
- **Serial by nature:** the LADMM iterations. Nothing about iteration $k+1$ can start before iteration k ends, so the wall time is iterations $\times$ ms/iteration and the only levers are the per-iteration cost (this note) and the iteration count (γ, the time grid, the initial guess).

7.4 Communication

With one GPU there is no device-to-device traffic. Host–device traffic per solve: the initial upload of the problem and precompute (tens of MB), one download of $9 \cdot$ `check_every` doubles per chunk (a few kB), and the final download of the solution. On a GH200 that path is NVLink-C2C at 900 GB/s, so even the eager code's per-iteration syncs were latency, not bandwidth. The suite's other communication is the file system: 2–6 MB of `.npz` per run.

7.5 Multiple GPUs

The four GPUs of a node hold 390 GB together. Splitting the arrays across them buys memory and bandwidth; it costs communication at every operator that couples the distributed axis.

Decomposition. Slabs along t are the natural choice: the spatial DCTs are then entirely local, the time-direction operators (A_t, $A_t^\top$, R_t, $R_t^\top$) need a one-plane halo exchange (16 MB at 128^3 in space), and only the per-mode time solve needs the whole t axis on one device: one all-to-all transpose of the mode array into t-complete pencils, solve, transpose back. Slabs along z move the same transpose to the z-DCT instead. The norms need an all-reduce of one scalar each. In JAX this is `jax.sharding` with `shard_map`; XLA emits the NCCL collectives.

Cost. Per iteration, two transposes of one float64 array plus halos. At 128^4 an array is 2.1 GB; over NVLink the two transposes take on the order of 10–15 ms against $\sim$40 ms of local compute per GPU, so about $3\times$ on four GPUs and the full node's memory. At 2D sizes (256^3: 134 MB per array, 7 ms per iteration) the transposes are $\sim$1 ms each and the launch overhead of the collectives is comparable, so the gain would be $1.5\times$ at best.

Verdict. Not useful for 2D: every 2D grid in this study fits and runs in minutes on one GPU. Useful in 3D for one reason, memory: 128^4 in float64 or `mixedc` does not fit on one GPU (Section 8) and fits comfortably on four. Stay within one node; across nodes the transposes would run over Slingshot and the iteration would be communication-bound. The engineering cost is moderate (the sharding annotations and one transpose in the projection) and should come after the 3D code has the single-GPU path of Sections 2–3.

8 What carries to 3D

The 3D tree (`python/3d`) still has the eager, list-based Thomas sweep and no compiled path. In order: register the 3D state and precompute as pytrees; port `ladmm_jit.py` unchanged (it is grid-agnostic); write the 3D DCT as three `tensordot` calls and the DCT-in-t mode solver with the same Woodbury correction (only $\lambda_{xy} \to \lambda_{xyz}$ changes); build all zero planes and initial iterates in `problem.rho0.dtype`; then enable `mixedc`. Per array, $n^4 \times 8$ bytes; with $\sim$55 array-equivalents (four state components) the single-GPU limit on a 98 GB GH200 is 112^4 in `mixedc` and 96^4 in float64; 128^3 in space fits with $n_t = 64$ or after dropping the references and the unused carry entries.

References

[1] JAX documentation: *How to think in JAX*, *JAX – The Sharp Bits*, `jax.lax.scan`, `jax.jit`. `https://jax.readthedocs.io`.

[2] W. W. Hager, Updating the inverse of a matrix, *SIAM Review* 31 (1989) 221–239.

[3] J. Makhoul, A fast cosine transform in one and two dimensions, *IEEE Trans. Acoust. Speech Signal Process.* 28 (1980) 27–34.

[4] S. Williams, A. Waterman, D. Patterson, Roofline: an insightful visual performance model for multicore architectures, *Comm. ACM* 52 (2009) 65–76.

[5] Y. Zhang, J. Cohen, J. D. Owens, Fast tridiagonal solvers on the GPU, *PPoPP* 2010.

[6] D. Goldberg, What every computer scientist should know about floating-point arithmetic, *ACM Computing Surveys* 23 (1991) 5–48.

[7] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002.

[8] P. Micikevicius et al., Mixed precision training, *ICLR* 2018, arXiv:1710.03740.

[9] E. Carson, N. J. Higham, Accelerating the solution of linear systems by iterative refinement in three precisions, *SIAM J. Sci. Comput.* 40 (2018) A817–A847.

[10] N. J. Higham, T. Mary, Mixed precision algorithms in numerical linear algebra, *Acta Numerica* 31 (2022) 347–414.

[11] A. Abdelfattah et al., A survey of numerical linear algebra methods utilizing mixed-precision arithmetic, *Int. J. High Perf. Comput. Appl.* 35 (2021) 344–369.